SUMMER TRAINING

PROJECT REPORT

(Term June-July 2025)

(Hostel Allotment System)

Submitted by

Shivangi Singh
Registration Number 12316998

Course Code PETV50

Under the Guidance of

Harjeet Kaur 12427

School of Computer Science and Engineering

CONTENTS OF THE REPORT

- Cover page
- Certificate
- Acknowledgement
- Table of Contents

CERTIFICATE

This is to certify that the Summer Training/Internship project report titled “HOSTEL ALLOTMENT SYSTEM” submitted by Shivangi Singh, Registration Number: 12316998, in partial fulfilment of the requirements for the course CSE343 / CSE443, is a record of her original work carried out during the Summer Term June–July 2025 under my guidance.

This project work is a part of the academic curriculum of the School of Computer Science and Engineering, and has not been submitted elsewhere for any other purpose.

Project Guide

Harjeet Kaur

Assistant Professor

School of Computer Science and Engineering

Lovely Professional University

Acknowledgement

I am extremely grateful to the School of Computer Science and Engineering for providing me the opportunity to work on this project titled "Hostel Management System using C++." I would also like to express my deep gratitude to my mentor [Faculty Name], who guided me patiently throughout this journey. Their insights and suggestions helped me learn and apply Data Structures in a practical scenario.

Lastly, I would like to thank my peers and institution for their support and encouragement.

📚 Table of Contents

1. Chapter 1: Introduction
2. Chapter 2: Training Overview
3. Chapter 3: Project Details
4. Chapter 4: Implementation
5. Chapter 5: Results and Discussion
6. Chapter 6: Conclusion

Chapter 1: INTRODUCTION

Company Profile:

This project was developed under academic supervision at Lovely Professional University as part of the Summer Training for Data Structures & Algorithms (CSE343 / CSE443).

Overview of Training Domain:

The training focused on **Data Structures**, **Object-Oriented Programming**, and **File Handling in C++**, with hands-on experience to implement these concepts in real-life problems.

Objective of the Project:

To create a simple yet robust Hostel Management System capable of managing student room allocations, tracking availability, and enabling room cancellation with an auto-update waiting list.

Chapter 2: TRAINING OVERVIEW

Tools & Technologies Used:

- **Language:** C++
- **IDE:** VS Code
- **Concepts:** OOP, File Handling, STL (vector, queue), Arrays

Areas Covered:

- Classes and Object-Oriented Concepts
- Arrays and Vectors
- Queues (FIFO for Waiting List)
- File I/O
- Conditional Logic and Control Flow

Daily/Weekly Summary:

- Week 1: Understanding STL, planning project logic
- Week 2: Coding core classes and room logic
- Week 3: Adding file handling and login features
- Week 4: Testing, debugging, and documentation

Chapter 3: PROJECT DETAILS

Title of the Project: Hostel Allocation System Using C++

Problem Definition:

Hostel administrators face challenges managing room assignments, handling cancellations, and ensuring fair distribution. This system automates all these tasks.

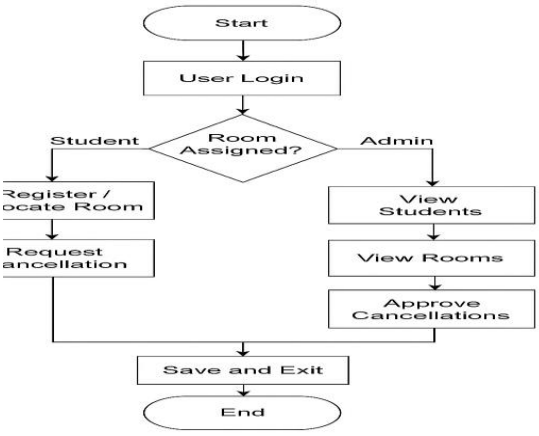
Scope & Objectives:

- Assign rooms based on availability.
- Track occupied/free rooms.
- Allow students to register/login.
- Handle cancellation requests and reassign rooms automatically.

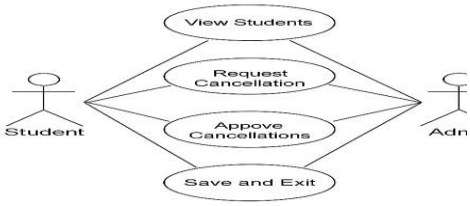
System Requirements:

- Windows OS
- C++ Compiler
- File I/O support

Architecture Diagram:



UML Diagram:



Chapter 4: IMPLEMENTATION

Tools Used:

- **C++ STL:** For managing data structures
- **File Handling:** Persistent data storage

Methodology:

- **Linear search** for login validation
- **Boolean arrays** to track room allocation
- **Queue** to manage waiting list
- **Vectors** for dynamic student data

Modules:

- Student Login/Registration
- Room Allocation
- Cancellation Request
- Admin Panel
- Waiting List Management

Code Snippets:

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <fstream>
using namespace std;

const int ROOM_START = 101;
const int ROOM_END = 200;
const int TOTAL_ROOMS = ROOM_END - ROOM_START + 1;

class Student {
public:
string id;
string email;
string password;
int roomNo;
bool cancelRequested;
Student(string i, string e, string p) {
    id = i;
    email = e;
    password = p;
    roomNo = -1;
    cancelRequested = false;
}
```

```cpp
for (int i = 0; i < students.size(); ++i) {
    Student &s = students[i];
    if (s.id == id && s.email == email && s.password == password) {
        cout << "Login successful.\n";
        if (s.roomNo != -1) {
            cout << "You have already been assigned Room No: " << s.roomNo <<
                endl;
            char c;
            cout << "Do you want to cancel it? (y/n): ";
            cin >> c;
            if (c == 'y' || c == 'Y') {
                s.cancelRequested = true;
                cout << "Room cancellation request sent.\n";
            }
        } else {
            allocateRoom(s);
        }
        return;
    }
}
cout << "Student not found. Registering...\n";
Student newStudent(id, email, password);
students.push_back(newStudent);
allocateRoom(students.back());
```

```cpp
void adminLogin() {
string user, pass;
cout << "\n--- Admin Login ---\n";
cout << "Username: "; cin >> user;
cout << "Password: "; cin >> pass;

if (user == "admin" && pass == "admin123") {
    int choice;
    while (true) {
        cout << "\n1. View Students\n2. View Rooms\n3. Approve
            Cancellations\n4. Logout\nEnter choice: ";
        cin >> choice;
        switch (choice) {
            case 1: viewAllStudents(); break;
            case 2: viewAllRooms(); break;
            case 3: approveCancellations(); break;
            case 4: return;
            default: cout << "Invalid choice.\n";
        }
    }
} else {
    cout << "Invalid admin credentials.\n";
}
```

```cpp
void viewAllRooms() {
cout << "\n--- Room Status ---\n";
for (int i = 0; i < TOTAL_ROOMS; ++i) {
cout << "Room " << (i + ROOM_START) << ": " << (rooms[i] ? "Occupied" : "Free"
    ) << endl;
}
}

void approveCancellations() {
cout << "\n--- Approving Cancellations ---\n";
for (Student &s : students) {
if (s.cancelRequested && s.roomNo != -1) {
cout << "Cancelling room " << s.roomNo << " for student " << s.id << endl;
rooms[s.roomNo - ROOM_START] = false;
s.roomNo = -1;
s.cancelRequested = false;

        if (!waitingList.empty()) {
            int index = waitingList.front();
            waitingList.pop();
            allocateRoom(students[index]);
        }
    }
```

Chapter 5: RESULTS AND DISCUSSION

Output:

- Properly allocates and deallocates rooms.
- Maintains student records persistently.
- Reassigns freed rooms to students in queue.

```
=== HOSTEL MANAGEMENT SYSTEM ===
1. Student Login
2. Admin Login
3. Exit
Enter choice: 1

--- Student Login ---
Enter ID: 123
Enter Email: yhuks
Enter Password: rf4
Student not found. Registering...

Enter your preferred room number (101 to 200): 102
Room 102 allocated successfully!
```

```
=== HOSTEL MANAGEMENT SYSTEM ===
1. Student Login
2. Admin Login
3. Exit
Enter choice: 2

--- Admin Login ---
Username: admin
Password: admin123

1. View Students
2. View Rooms
3. Approve Cancellations
4. Logout
Enter choice: 1

--- Student List ---
ID: 123, Email: yghgf, Room: 103, Cancel Requested: No
```

Challenges:

- File handling logic was tricky initially.
- Ensuring real-time updates to room status required testing.

Learnings:

- How to work with vectors, arrays, and queues together.
- Real-world application of file I/O in project development.
- Object-oriented design in C++.

Chapter 6: CONCLUSION

Summary:

This project enhanced my skills in C++ and taught me how to build structured systems using data structures and file handling. It solves a real problem using an academic approach and demonstrates how theoretical DSA concepts apply practically.